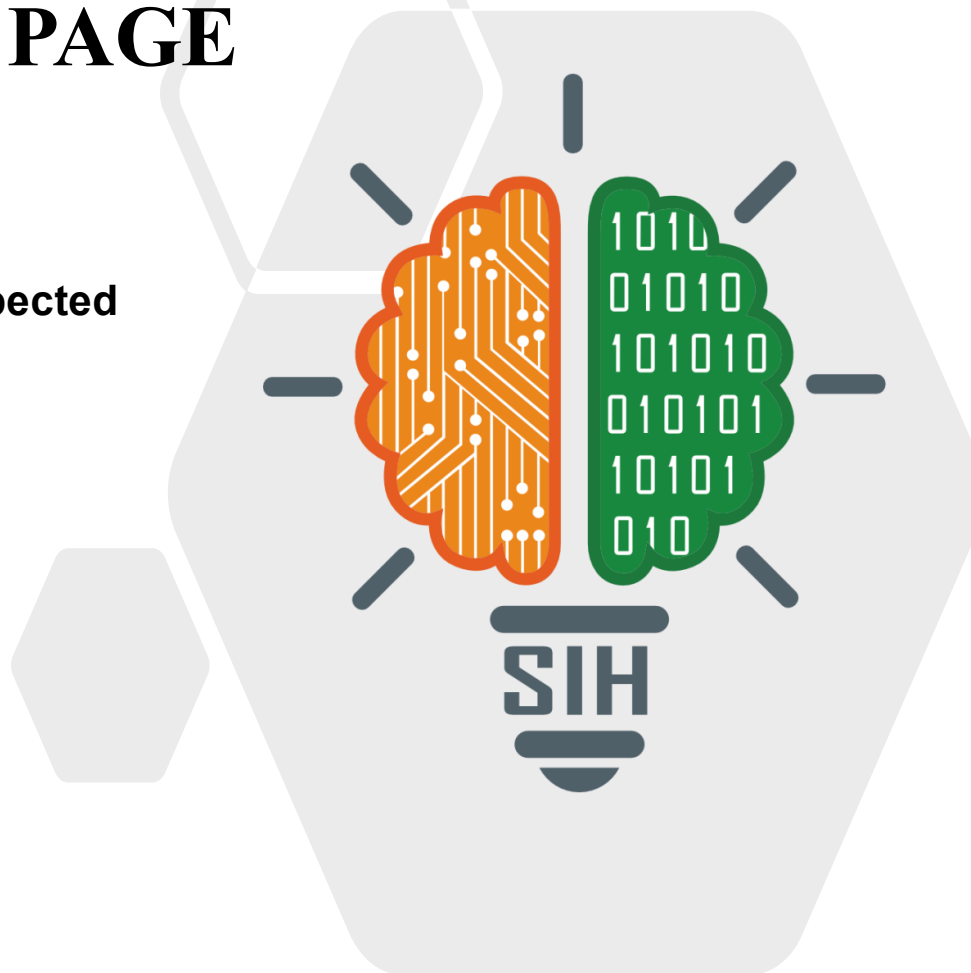


SMART INDIA HACKATHON 2026



TITLE PAGE

- **Problem Statement ID – 26028**
- **Problem Statement Title- Dynamic Forecast of Expected Time of Arrival (ETA) for Coaching Trains**
- **Theme- Smart Automation**
- **PS Category- Software**
- **Team ID-**
- **Team Name (Registered on portal)- Data Dynamos**



PROBLEM STATEMENT



Dynamic Forecast of Expected Time of Arrival(ETA) for coaching trains



❖ Proposed Solution: Dynamic ETA Prediction for Indian Railways

- RailPulse predicts real arrival times using live delay, weather, congestion and station history - recalculating continuously as new data arrives, not a static timetable
- Replaces static schedule + recovery-margin ETAs with a live, self-correcting prediction, cutting uncertainty for passengers, station staff and control rooms
- Hybrid physics + ML model with confidence intervals and a transparent "why did ETA change" breakdown by weather / congestion / dwell / carried-over delay - not a black box

LIVE DEMO: REAL PROTOTYPE RESPONSE



Train 12000 approaching Kishanganj (KIR) - actual response from working system

BEFORE

3.3 min

Predicted Delay

Explanation: Carried over delay **+2.0 min**

AFTER LIVE UPDATE

14.4 min

Predicted Delay

Explanation: Carried over delay **+15.0 min**

+11.1 min recalculated instantly

SYSTEM ARCHITECTURE



Data Sources

GPS, Timetable, Delays, Weather, Congestion



Ingestion & Features

Kafka, Redis Cache



Prediction Engine

XGBoost Residual Model



Serving API

FastAPI



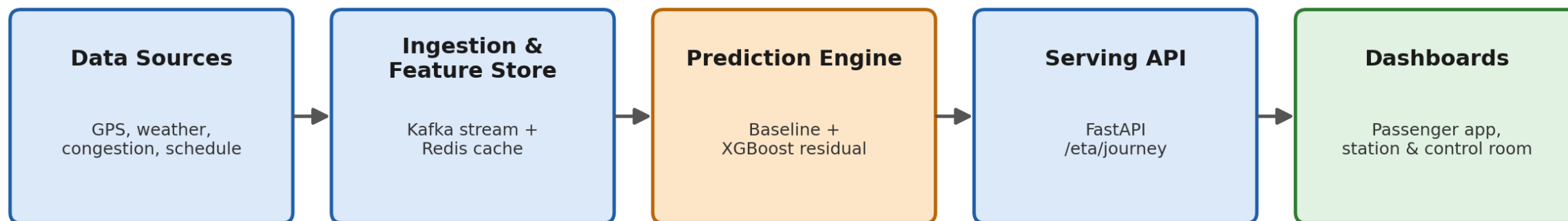
Dashboards

React, Live Updates

TECHNICAL APPROACH



- Technologies: FastAPI backend, XGBoost residual model, PostgreSQL + Redis, React dashboard, Kafka (production-scale ingestion)
- Hybrid approach: a physics-based baseline (schedule + carried-over delay) corrected by an ML model trained on weather, congestion and historical dwell time
- Every new location/delay update triggers an automatic recalculation, with a structured explanation of exactly what changed and why
- Result: 58.7% lower prediction error (2.03 min MAE) vs a naive "delay stays constant" baseline (4.91 min MAE), measured on held-out test data



FEASIBILITY AND VIABILITY

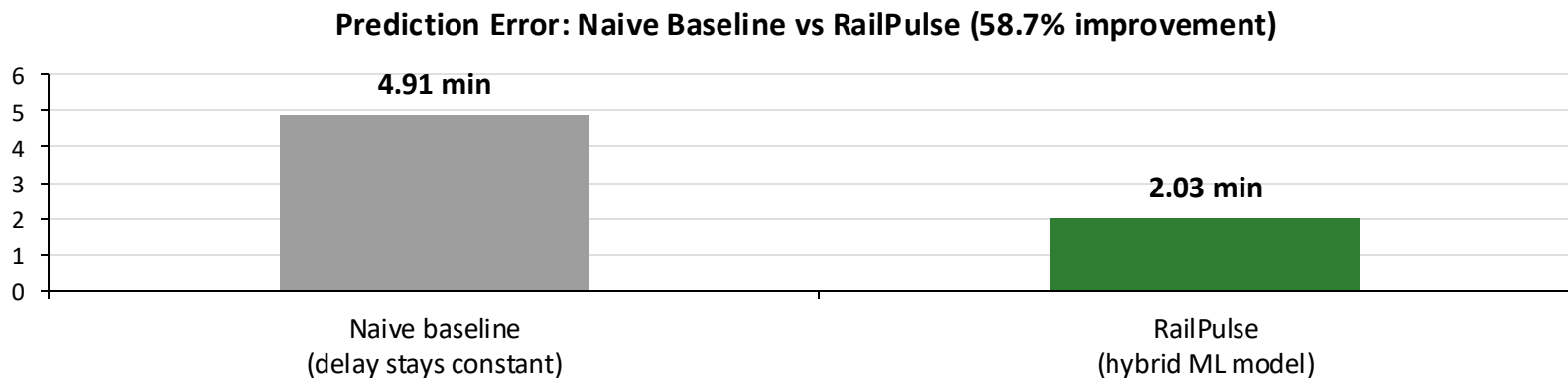


- Working prototype already built and tested end-to-end: data simulator, FastAPI backend, trained XGBoost model, live React dashboard
- Response schema matches a real NTES/GPS feed exactly, so swapping in live Indian Railways data later needs zero pipeline changes
- Key risk: real-time IR GPS/NTES data is not publicly available for prototyping, and connectivity is patchy in remote sections
- Strategy: schema-compatible simulator today for a seamless data swap later; GSM/LoRa fallback alerts for low-connectivity sections; phased rollout starting on one high-traffic corridor

IMPACT AND BENEFITS



- Passengers get trustworthy live arrival times instead of a static, often-wrong schedule
- Station staff get earlier visibility for platform allocation, crew scheduling and cleaning operations
- Control rooms see cascading delays forming earlier on multi-day journeys, enabling proactive resourcing
- Measured benefit: prediction error cut by 58.7% vs current static-schedule approach (see chart)



RESEARCH AND REFERENCES



- NTES - Indian Railways real-time train tracking: enquiry.indianrail.gov.in/ntes
- IEEE - "Real-Time Passenger Train Delay Prediction Using Machine Learning" (RF / GBM / MLP comparison)
- IEEE - "Forecasting and Analysis of Train Delays and Impact of Weather Data using ML" (India-specific study)
- XGBoost documentation: xgboost.readthedocs.io | FastAPI documentation: fastapi.tiangolo.com
- Prototype: GitHub - [Your GitHub Link] | Demo video - [Your Demo Video Link]

FUTURE SCOPE: PREDICTION TO PREVENTION



1. DATA & INFRASTRUCTURE

- Replace the local simulator with a real or semi-real data feed — pursue access to NTES/CRIS sample data or partner with Railways-side mentors for a sandboxed data source
- Stand up the Kafka + Redis pipeline for real (currently architected, not implemented) so ingestion is genuinely live, not simulated
- Deploy backend + frontend to real infrastructure instead of localhost, so it's demoable from anywhere

2. MODEL

- Expand from one corridor (30 trains) to multiple zones/routes to test generalization
- Explore a sequence model (LSTM) as a second approach and compare against XGBoost on accuracy vs latency trade-offs
- Add anomaly detection for rare events (derailment-adjacent delays, major weather disruptions) that the current model doesn't specifically handle

3. PRODUCT

- Build the passenger-facing mobile view (currently only the control-room dashboard exists)
- Add SMS/push notification alerts for stations with poor connectivity, not just a live dashboard
- Add authentication and role-based access for the control room dashboard (currently open, prototype-only)

4. VALIDATION

- Run a small pilot comparison against real historical NTES data (even scraped/public data) to validate our MAE numbers against real-world variance, not just our synthetic dataset
- Load-test the API for concurrent requests across thousands of trains, not just 30

TEAM MEMBERS



- Shivam Dutta – ADTU/0/2025-27/MCAM/011
- Mriganka Saikia - ADTU/0/2025-27/MCAM/012
- Abhijit Nath - ADTU/0/2025-27/MCAM/014
- Pratistha Hazarika - ADTU/0/2025-27/MCAM/033
- Khusi Debbarma - ADTU/0/2026-28/MCAM/030
- Sudipta Kashyap - ADTU/0/2024-28/BCSM/048